BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 1

Roll no: 51 PRN No: 12412161 Name of Student: Utkarsh Salunkhe

Subject Name : Deep Learning

Problem Statement : Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Algorithm :

1. Download the Tomato Leaf Disease dataset from Kaggle.
2. Extract the dataset and identify the ten disease/healthy classes.
3. Resize all images to 128 × 128 pixels.
4. Normalize pixel values from 0–255 to 0–1.
5. Divide the images into 80% training and 20% validation sets.
6. Construct a CNN with three convolutional and max-pooling blocks.
7. Flatten the extracted features.
8. Add a Dense layer and Dropout layer.
9. Use a Softmax output layer for the ten-class classification.
10. Compile the model using Adam optimizer and categorical cross-entropy.
11. Train the CNN for 15 epochs.
12. Evaluate the model using validation accuracy and loss.
13. Generate a classification report and confusion matrix.
14. Upload a new tomato leaf image.
15. Preprocess the image and pass it through the trained CNN.
16. Display the predicted disease and prediction confidence.

Code and output :

```
[17]
✓ 0s
import tensorflow as tf

print("TensorFlow Version:", tf.__version__)
print("GPU:", tf.config.list_physical_devices('GPU'))

TensorFlow Version: 2.20.0
GPU: []
```

```
[14]
✓ 0s
import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import os
import zipfile
import requests

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing import image

from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix

import seaborn as sns
```

```
[15]
✓ 6s
url = "https://www.kaggle.com/api/v1/datasets/download/kaustubhb999/tomatoleaf"

response = requests.get(url)

print("Status code:", response.status_code)

with open("/content/tomatoleaf.zip", "wb") as f:
    f.write(response.content)

print("Dataset downloaded successfully!")

file_size = os.path.getsize("/content/tomatoleaf.zip") / (1024 * 1024)

print("File size:", round(file_size, 2), "MB")

Status code: 200
Dataset downloaded successfully!
File size: 178.87 MB
```

```
[16]
✓ 3s
dataset_path = "/content/tomato_dataset"

os.makedirs(dataset_path, exist_ok=True)

with zipfile.ZipFile(
    "/content/tomatoleaf.zip",
    "r"
) as zip_ref:

    zip_ref.extractall(dataset_path)

print("Dataset extracted successfully!")

Dataset extracted successfully!
```

```
[17]
✓ 0s
for root, dirs, files in os.walk(dataset_path):

    level = root.replace(
        dataset_path, ""
    ).count(os.sep)

    if level <= 2:

        print(
            "  " * level +
            os.path.basename(root) +
            "/"
        )

Show hidden output
```

```
[18]
✓ 0s
image_extensions = (
    ".jpg",
    ".jpeg",
    ".png"
)

image_folders = []

for root, dirs, files in os.walk(dataset_path):

    image_count = sum(
        file.lower().endswith(image_extensions)
        for file in files
    )

    if image_count > 0:
```

```
candidate_dirs = []

for root, dirs, files in os.walk(dataset_path):

    class_count = 0

    for directory in dirs:

        folder_path = os.path.join(
            root,
            directory
        )

        if not os.path.isdir(folder_path):
            continue

        image_count = sum(
            file.lower().endswith(image_extensions)
            for file in os.listdir(folder_path)
            if os.path.isfile(
                os.path.join(
                    folder_path,
                    file
                )
            )
        )

        if image_count > 0:

            class_count += 1

    if class_count >= 2:

        candidate_dirs.append(
```

```
DATASET_DIR = candidate_dirs[0][0]

print("Dataset directory:")
print(DATASET_DIR)
```

```
Dataset directory:
/content/tomato_dataset/tomato/train
```

```
classes = sorted([
    folder
    for folder in os.listdir(DATASET_DIR)
    if os.path.isdir(
        os.path.join(
            DATASET_DIR,
            folder
        )
    )
])

print("Number of classes:", len(classes))

print("\nClasses:")

for i, class_name in enumerate(classes):

    print(
        i,
        ":",
        class_name
    )
```

```
Number of classes: 10
```

```
plt.bar(
    df["Disease"],
    df["Number of Images"]
)

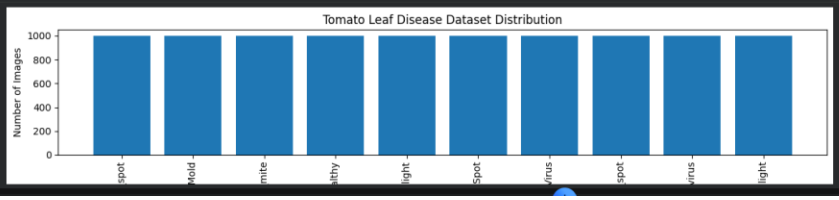
plt.xlabel("Disease Class")
plt.ylabel("Number of Images")

plt.title(
    "Tomato Leaf Disease Dataset Distribution"
)

plt.xticks(
    rotation=90
)

plt.tight_layout()

plt.show()
```



```
IMG_HEIGHT = 128
IMG_WIDTH = 128

BATCH_SIZE = 32

print("Image Height:", IMG_HEIGHT)
print("Image Width:", IMG_WIDTH)
print("Batch Size:", BATCH_SIZE)
```

```
Image Height: 128
Image Width: 128
Batch Size: 32
```

```
datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.20
)
```

```
train_data = datagen.flow_from_directory(

    DATASET_DIR,

    target_size=(
        IMG_HEIGHT,
        IMG_WIDTH
    ),

    batch_size=BATCH_SIZE,

    class_mode="categorical",

    subset="training",

    shuffle=True
```

Terminal 09:46 Python 3

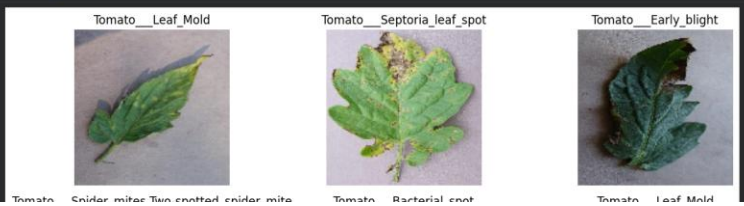
```
validation_data = datagen.flow_from_directory(

    DATASET_DIR,
    target_size=(
        IMG_HEIGHT,
        IMG_WIDTH
    ),
    batch_size=BATCH_SIZE,
    class_mode="categorical",
    subset="validation",
    shuffle=False
)
```

```
Found 2000 images belonging to 10 classes.
```

```
class_indices = train_data.class_indices
class_names = list(
    class_indices.keys()
)
num_classes = len(
    class_names
)
print(
    "Number of classes:",
    num_classes
)
print("\nClass Mapping:")
for class_name, index in class_indices.items():
    print(
        index,
        ":",
        class_name
    )
```

```
for i in range(9):
    plt.subplot(
        3,
        3,
        i + 1
    )
    plt.imshow(
        images[i]
    )
    class_index = np.argmax(
        labels[i]
    )
    plt.title(
        class_names[class_index]
    )
    plt.axis("off")
plt.tight_layout()
plt.show()
```



```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 128, 128, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 128)	3,211,392
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 3,305,930 (12.61 MB)
Trainable params: 3,305,930 (12.61 MB)
Non-trainable params: 0 (0.00 B)

```
model.compile(

    optimizer="adam",

    loss="categorical_crossentropy",

    metrics=["accuracy"]
)
```

```
EPOCHS = 5

history = model.fit(

    train_data,

    validation_data=validation_data,

    epochs=EPOCHS
)
```

```
Epoch 1/5
250/250 ━━━━━━━━━━━━━━━━━━━━ 309s 1s/step - accuracy: 0.5173 - loss: 1.3855 - val_accuracy: 0.7400 - val_loss: 0.7919
Epoch 2/5
250/250 ━━━━━━━━━━━━━━━━━━━━ 334s 1s/step - accuracy: 0.7145 - loss: 0.8160 - val_accuracy: 0.7950 - val_loss: 0.5782
Epoch 3/5
250/250 ━━━━━━━━━━━━━━━━━━━━ 306s 1s/step - accuracy: 0.7713 - loss: 0.6640 - val_accuracy: 0.8615 - val_loss: 0.4216
Epoch 4/5
250/250 ━━━━━━━━━━━━━━━━━━━━ 286s 1s/step - accuracy: 0.8067 - loss: 0.5420 - val_accuracy: 0.8710 - val_loss: 0.4107
Epoch 5/5
250/250 ━━━━━━━━━━━━━━━━━━━━ 301s 1s/step - accuracy: 0.8406 - loss: 0.4646 - val_accuracy: 0.8990 - val_loss: 0.3223
```

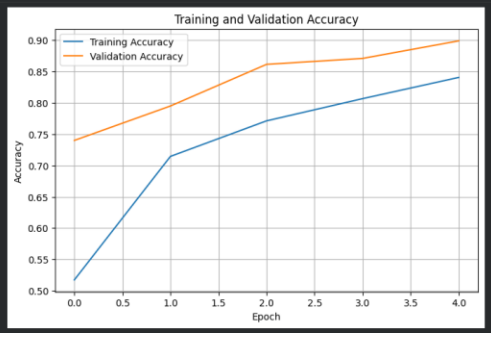
```
plt.figure(figsize=(8, 5))
```

```
)

plt.legend()

plt.grid()

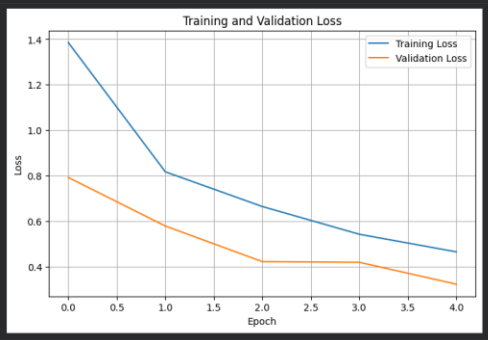
plt.show()
```



```
plt.legend()

plt.grid()

plt.show()
```




```
validation_data.reset()
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 20s 319ms/step
```

```
report = classification_report(

    true_classes,

    predicted_classes,

    target_names=class_names
)

print(report)
```

```
                                          precision    recall  f1-score   support

                      Tomato___Bacterial_spot       0.89      0.92      0.91       200
                        Tomato___Early_blight       0.78      0.82      0.80       200
                         Tomato___Late_blight       0.87      0.86      0.86       200
                           Tomato___Leaf_Mold       0.92      0.93      0.93       200
                  Tomato___Septoria_leaf_spot       0.92      0.86      0.89       200
Tomato___Spider_mites Two-spotted_spider_mite       0.87      0.93      0.90       200
                         Tomato___Target_Spot       0.86      0.92      0.89       200
       Tomato___Tomato_Yellow_Leaf_Curl_Virus       0.95      0.91      0.93       200
                 Tomato___Tomato_mosaic_virus       0.95      0.97      0.96       200
                             Tomato___healthy       0.99      0.94      0.97       200

                                     accuracy                           0.90      2000
                                    macro avg       0.90      0.90      0.90      2000
                                 weighted avg       0.90      0.90      0.90      2000
```

```
cm = confusion_matrix(

    true_classes,
```

